

7. Code speedup

7.1. Why computational performance matters

Modern computational physics problems often involve extremely large numerical simulations and data sets. Examples include

- magnetohydrodynamic simulations,
- Monte Carlo methods,
- N -body simulations,
- matrix operations,
- particle-based methods,
- finite-difference and finite-volume solvers.

The problems differ substantially in memory requirements and computational demand. A few examples are shown in table 7.1.

Table 7.1.: Typical sizes of modern computational physics problems. The numbers are rough order-of-magnitude estimates intended to illustrate the enormous computational requirements of contemporary simulations.

Problem	Typical size	Memory	Output data	Typical runtime
2D Ising model	1024^2 spins	~ 1 MB	~ 100 MB	minutes
3D fluid simulation	512^3 cells	~ 10 GB	~ 1 TB	hours–days
3D MHD galaxy simulation	1024^3 cells	~ 100 GB	~ 100 TB	weeks
Cosmological simulation	10^{10} particles	~ 10 TB	~ 1 PB	months
Molecular dynamics	10^9 particles	~ 100 GB	~ 10 TB	days–weeks
Monte Carlo transport	10^{12} trajectories	small per particle	~ 10 TB	days
Climate simulation	$> 10^9$ grid cells	~ 1 TB	~ 1 PB	months

7.1.1. Moore’s law and the end of frequency scaling

Historically, the increase in computational power was driven by *Moore’s law*, which states that the number of transistors on a processor roughly doubles every two years. For many decades, this translated directly into faster CPUs with increasing clock frequencies. As a consequence, older programs often became faster automatically on newer

7. Code speedup

hardware without requiring modifications. However, around the mid-2000s, processor clock frequencies largely stopped increasing because of

- power consumption,
- heat dissipation,
- memory bandwidth limitations.

Modern processors therefore no longer gain most of their performance from faster individual cores, but instead from

- multiple CPU cores,
- vector instructions,
- simultaneous multithreading,
- GPU accelerators,
- distributed computing clusters.

As a result, scientific software must increasingly exploit *parallelism* in order to benefit from modern hardware. This development fundamentally changed scientific computing and related software development: *Performance improvements no longer occur automatically. Modern codes must be explicitly parallelized.*

7.1.2. Algorithms versus brute-force parallelization

An important lesson is:

A better algorithm is often more important than parallelization.

For example, replacing an $\mathcal{O}(N^2)$ algorithm by an $\mathcal{O}(N \log N)$ algorithm can yield orders-of-magnitude improvements, often larger than the gain from adding more processors. A few examples include:

- tree methods instead of direct N -body summation,
- FFT-based solvers instead of direct convolutions,
- sparse matrix methods instead of dense matrix operations.

Efficient scientific computing therefore combines:

- better algorithms,
- optimized memory access,
- vectorization,
- parallelization,
- hardware acceleration.

7.1.3. Example: Reducing the cost of gravitational forces: grouping distant particles

Problem

A direct gravitational N -body calculation computes the acceleration of every particle from every other particle:

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}. \quad (7.1)$$

For N particles, this requires approximately

$$N(N-1) \sim \mathcal{O}(N^2) \quad (7.2)$$

pairwise interactions.

However, if the spatial extent of group of particles is small compared to its distance from particle i , the detailed positions of the particles inside the group are not important. The entire group can be approximated by a single mass located at its center of mass.

Consider a group of particles with total mass

$$M = \sum_{j \in \text{cell}} m_j \quad (7.3)$$

and center of mass

$$\mathbf{x}_{\text{cm}} = \frac{1}{M} \sum_{j \in \text{cell}} m_j \mathbf{x}_j. \quad (7.4)$$

The exact acceleration from this group is

$$\mathbf{a}_{i,\text{cell}} = G \sum_{j \in \text{cell}} m_j \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3}. \quad (7.5)$$

If the group is sufficiently far away, we approximate this by

$$\mathbf{a}_{i,\text{cell}} \approx GM \frac{\mathbf{x}_{\text{cm}} - \mathbf{x}_i}{|\mathbf{x}_{\text{cm}} - \mathbf{x}_i|^3}. \quad (7.6)$$

Hierarchical grouping and tree methods

The basic idea behind modern gravitational solvers is to organize particles hierarchically according to their spatial positions. Instead of storing all particles in one large list, the simulation volume is recursively subdivided into smaller regions (see Fig. 7.1):

- the entire simulation domain forms the root node,
- each region is subdivided into smaller subregions,

7. Code speedup

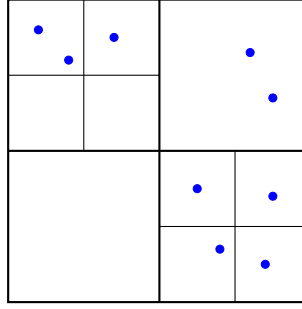


Figure 7.1.: Example of a hierarchical spatial subdivision in two dimensions. Dense regions are recursively subdivided into smaller cells, forming a tree structure.

- the subdivision continues until only a few particles remain per cell.

This hierarchical subdivision naturally forms a *tree structure*. In two dimensions, each square cell is subdivided into four smaller cells, forming a *quadtree*. In three dimensions, each cube is subdivided into eight smaller cubes, forming an *octree*.

The force calculation then proceeds as follows:

- nearby cells are opened and examined in more detail,
- distant cells are treated as a single center-of-mass interaction.

The tree is therefore traversed recursively:

1. start at the root node,
2. test whether a cell is sufficiently far away,
3. if yes: use the center-of-mass approximation,
4. if no: open the cell and examine its children.

This procedure is illustrated schematically below.

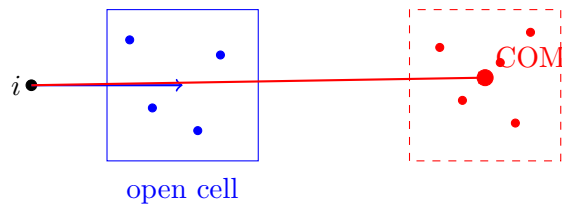


Figure 7.2.: Nearby cells are opened and resolved in detail, while distant cells are approximated by a single center-of-mass interaction.

The opening criterion compares the size of a cell to its distance from the particle:

$$\frac{l}{r} < \theta, \quad (7.7)$$

where l is the size of the cell, r is the distance to the cell center, and θ is the opening angle parameter. If the criterion is satisfied, the entire cell is approximated by its total mass and center of mass. Otherwise, the cell is opened and its subcells are examined individually. Small values of θ give more accurate forces but require more work because more cells/branches of the tree need to be opened and more computation needs to be done as direct sum. Large values of θ are faster but less accurate.

This hierarchical procedure dramatically reduces the number of interactions. Instead of computing forces between all particle pairs, distant groups are combined into a small number of effective interactions. The resulting computational cost scales approximately as

$$\mathcal{O}(N \log N) \tag{7.8}$$

instead of

$$\mathcal{O}(N^2). \tag{7.9}$$

The essential idea is therefore *Resolve nearby structure accurately, but approximate distant structure collectively*.

The gain in different scaling is often a much larger than what could be achieved by parallelizing the direct summation alone. For example, increasing the particle number from $N = 10^3$ to $N = 10^6$ increases the direct cost by roughly

$$\left(\frac{10^6}{10^3}\right)^2 = 10^6, \tag{7.10}$$

whereas the tree-code cost grows approximately as

$$\frac{10^6 \log 10^6}{10^3 \log 10^3} \sim 2 \times 10^3. \tag{7.11}$$

7.2. Measuring runtime and speedup

Let T_p denote the runtime of a program using p processors or threads. The speedup is defined as

$$S(p) = \frac{T_1}{T_p}. \tag{7.12}$$

An ideal parallel program would satisfy

$$S(p) = p, \tag{7.13}$$

meaning that doubling the number of processors halves the runtime. The parallel efficiency is defined as

$$E(p) = \frac{S(p)}{p}. \tag{7.14}$$

An efficiency of

$$E(p) = 1 \tag{7.15}$$

corresponds to perfect scaling.

7. Code speedup

7.2.1. Amdahl's law

In practice, programs contain serial parts that cannot be parallelized. Assume that a fraction f of the code is strictly serial, while the remaining fraction $(1 - f)$ can be parallelized perfectly. The total runtime becomes

$$T_p = fT_1 + \frac{(1 - f)T_1}{p}. \quad (7.16)$$

The resulting speedup is

$$S(p) = \frac{1}{f + \frac{1-f}{p}}. \quad (7.17)$$

Even for infinitely many processors,

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{f}. \quad (7.18)$$

Thus, a serial fraction of only 10% limits the maximum speedup to

$$S_{\max} = 10. \quad (7.19)$$

This result is known as *Amdahl's law* and illustrates why removing serial bottlenecks is essential for scalable parallel programs.

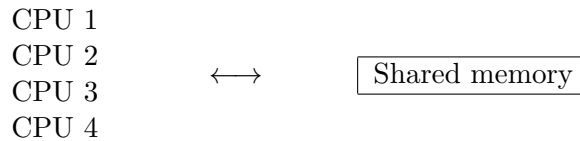
7.3. Shared and distributed memory

Parallel computers or software concepts can be classified into two major architectures:

- shared-memory systems,
- distributed-memory systems.

7.3.1. Shared-memory systems

In a shared-memory architecture, all processors access the same physical memory.



Parallelization is typically implemented using *threads*. Each thread executes part of the program while accessing the same arrays and variables.

Advantages:

- simple communication,

- fast data exchange,
- easy parallelization of loops.

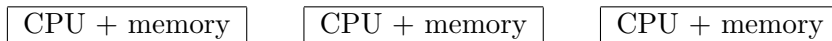
Disadvantages:

- limited by the memory of one machine,
- limited scalability (only up to number of *local* number of cores),
- race conditions may occur (several threads want to write to *same* data *simultaneously*).

A common shared-memory programming model is OpenMP.

7.3.2. Distributed-memory systems

In distributed-memory systems, each processor owns its own local memory.



Processors communicate explicitly through messages sent over a network.

Advantages:

- excellent scalability,
- large total memory,
- suitable for supercomputers.

Disadvantages:

- communication overhead,
- more complicated programming,
- explicit data exchange required.

The dominant distributed-memory standard is MPI (Message Passing Interface).

7.4. OpenMP

OpenMP is a shared-memory parallel programming standard for C, C++, and Fortran. It allows the parallelization of loops using compiler directives. A simple OpenMP loop has the form

```
#pragma omp parallel for
for (int i=0; i<N; i++) {
    a[i] = f(i);
}
```

The iterations of the loop are distributed among multiple threads.

7. Code speedup

7.4.1. Advantages of OpenMP

- simple to add to existing codes,
- ideal for loop parallelism,
- efficient on multi-core CPUs.

7.4.2. Challenges

- race conditions,
- synchronization overhead,
- load balancing,
- limited to one shared-memory node.

7.4.3. Race conditions

A race condition occurs when multiple threads modify the same variable simultaneously. An example is the computation of the sum:

```
sum += a[i];
```

Different threads may overwrite each other's updates. OpenMP therefore provides *reductions*:

```
#pragma omp parallel for reduction(+:sum)
```

7.5. MPI

MPI (Message Passing Interface) is the dominant standard for distributed-memory parallelization. Each MPI process has:

- its own memory,
- its own variables,
- a unique process ID called the rank.

Typical MPI programs begin with:

```
MPI_Init(...)
```

and end with:

```
MPI_Finalize(...)
```


7.5.1. Communication

MPI processes communicate explicitly using messages.

Common examples are

- `MPI_Send`,
- `MPI_Recv`,
- `MPI_Bcast`,
- `MPI_Reduce`.

7.5.2. Domain decomposition

Large simulations are typically decomposed spatially. For example, a two-dimensional grid may be split into subdomains:

Rank 0	Rank 1
Rank 2	Rank 3

Each process computes only its local domain and exchanges boundary information with neighboring processes.

7.6. Parallelization in Python

Python is widely used in scientific computing because of its simplicity and large ecosystem. However, pure Python loops are often slow because Python is an interpreted language. Several approaches exist to accelerate Python codes.

7.6.1. Vectorization with NumPy

NumPy performs array operations in optimized compiled code. Instead of writing

```
for i in range(N):
    a[i] = x[i]*x[i]
```

one writes

```
a = x*x
```

The advantages are a concise syntax, very high performance, internally optimized C routines. For many scientific applications, NumPy vectorization already provides substantial speedups.

7. Code speedup

7.6.2. Multiprocessing

Python's `multiprocessing` module launches multiple independent Python processes.

The advantages are:

- it bypasses the global interpreter lock (GIL),
- it is a straightforward parallel execution on CPUs.

Disadvantages are:

- communication overhead,
- memory duplication,
- process startup costs.

7.6.3. Threading and the GIL

Python threads are limited by the Global Interpreter Lock (GIL), which allows only one Python thread to execute Python bytecode at a time. As a consequence:

- threading is often ineffective for CPU-bound Python loops,
- threading is useful for I/O-bound tasks.

Examples of I/O-bound tasks:

- file access,
- network communication,
- waiting for external processes.

7.6.4. Numba

Numba accelerates Python functions through just-in-time (JIT) compilation.

A function can be compiled using

```
@njit
```

at the beginning of the function as in the following example

```
@njit
def f(x):
    s = 0.0
    for i in range(x.size):
        s += x[i]*x[i]
    return s
```

Numba can also parallelize loops:

```
@njit(parallel=True)

with
prange(...)
```

instead of `range(...)`.

The advantages are

- very large speedups,
- minimal code changes,
- excellent for numerical loops.

The downsides is that numba can only compile code that does not include too unspecific python constructs in terms of data types. Lists can have mixed data types and are thus not possible in numba.

7.6.5. MPI in Python: mpi4py

The package `mpi4py` provides MPI functionality in Python, which looks like

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
```

Typical applications are distributed Monte Carlo simulations, domain decomposition, large-scale HPC simulations. The parallel programs are typically launched using:

```
mpirun -n 4 python script.py
```

7.6.6. GPU acceleration

Graphics Processing Units (GPUs) contain thousands of lightweight processing cores and can accelerate massively parallel workloads. Python ecosystems for GPU support include

- CuPy,
- JAX,
- PyTorch.

GPUs are particularly efficient for:

- matrix operations,

7. Code speedup

- deep learning,
- massively parallel numerical kernels.

In many cases the bottleneck can be data transfer between CPU and GPU memory. Also, GPUs rely on performing the *same operations* on many tasks. If the problem does not factorize well, the GPU performance suffers.

7.7. Practical optimization strategy

A useful strategy for optimizing scientific codes is:

1. improve the algorithm,
2. use vectorization,
3. use compiled kernels (Numba/Cython),
4. parallelize,
5. optimize communication,
6. consider GPU acceleration.

Parallelization is therefore only one part of computational optimization. In practice, the largest gains often arise from:

- removing unnecessary work,
- improving memory access patterns,
- choosing better algorithms.

A. Mathematics

A.1. Random Numbers and Monte Carlo Error

A central concept of random numbers is that of *independent and identically distributed* (iid) random variables. Understanding iid variables is essential for deriving the convergence properties and error estimates of Monte Carlo integration. We introduce the concepts of expectation value, variance, and sample averages, and derive the characteristic Monte Carlo error scaling.

Independent and Identically Distributed Variables Consider a set of random variables

$$x_1, x_2, \dots, x_N. \quad (\text{A.1})$$

They are called

- **identically distributed** if all variables are drawn from the same probability distribution,
- **independent** if the value of one variable does not influence the others.

In Monte Carlo methods we usually assume that all sampled points are iid random variables.

Expectation Value For a random variable x with probability density function $p(x)$, the expectation value (or mean) is defined as

$$\mu = \langle x \rangle = \int x p(x) dx. \quad (\text{A.2})$$

The expectation value represents the average value obtained from infinitely many realizations of the random process.

Variance The variance measures the spread of the distribution around the mean,

$$\text{Var}(x) = \langle (x - \mu)^2 \rangle \quad (\text{A.3})$$

$$= \langle x^2 - 2\mu x + \mu^2 \rangle \quad (\text{A.4})$$

$$= \langle x^2 \rangle - 2\mu \langle x \rangle + \langle \mu^2 \rangle \quad (\text{A.5})$$

$$= \langle x^2 \rangle - \mu^2. \quad (\text{A.6})$$

The standard deviation is defined as

$$\sigma_x = \sqrt{\text{Var}(x)}. \quad (\text{A.7})$$

A. Mathematics

Sample Mean Suppose we draw N iid samples $x_1, x_2, \dots, x_N$. The sample mean is

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i. \quad (\text{A.8})$$

Because all samples are identically distributed,

$$\langle x_i \rangle = \mu. \quad (\text{A.9})$$

Therefore,

$$\langle \bar{x} \rangle = \frac{1}{N} \sum_{i=1}^N \langle x_i \rangle = \frac{1}{N} (N\mu) = \mu. \quad (\text{A.10})$$

Thus, the sample mean is an unbiased estimator of the true mean.

Variance of the Mean We now derive the variance of the sample mean. Starting from

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad (\text{A.11})$$

we compute

$$\text{Var}(\bar{x}) = \text{Var} \left(\frac{1}{N} \sum_{i=1}^N x_i \right). \quad (\text{A.12})$$

Variance scales quadratically under multiplication:

$$\text{Var}(aX) = a^2 \text{Var}(X), \quad (\text{A.13})$$

therefore

$$\text{Var}(\bar{x}) = \frac{1}{N^2} \text{Var} \left(\sum_{i=1}^N x_i \right). \quad (\text{A.14})$$

For independent random variables, variances add,

$$\text{Var} \left(\sum_{i=1}^N x_i \right) = \sum_{i=1}^N \text{Var}(x_i). \quad (\text{A.15})$$

Since all variables are identically distributed,

$$\text{Var}(x_i) = \sigma_x^2, \quad (\text{A.16})$$

which yields

$$\text{Var} \left(\sum_{i=1}^N x_i \right) = N\sigma_x^2. \quad (\text{A.17})$$

Substituting this result gives

$$\text{Var}(\bar{x}) = \frac{1}{N^2}(N\sigma_x^2) = \frac{\sigma_x^2}{N}. \quad (\text{A.18})$$

Hence,

$$\text{Var}(\bar{x}) = \frac{1}{N}\text{Var}(x) \quad (\text{A.19})$$

and the standard deviation becomes

$$\sigma_{\bar{x}} = \frac{\sigma_x}{\sqrt{N}} \quad (\text{A.20})$$

This is one of the most important results in statistics and Monte Carlo methods.

A.2. Joint probability distributions

In many applications we are interested in several random variables simultaneously. Examples include the position and velocity of a particle, temperature and pressure in a gas, or the spin states of neighboring particles in the Ising model. To describe such systems we introduce the concept of a *joint probability distribution*.

A.2.1. Joint probability for discrete random variables

Let X and Y be two discrete random variables. The *joint probability mass function* is defined as

$$P(X = x, Y = y). \quad (\text{A.21})$$

This quantity gives the probability that

- the random variable X takes the value x , and
- simultaneously the random variable Y takes the value y .

The joint probabilities must satisfy the normalization condition

$$\sum_x \sum_y P(X = x, Y = y) = 1. \quad (\text{A.22})$$

Example

Consider two binary random variables $X, Y \in \{0, 1\}$ with the following joint distribution:

X	Y	$P(X, Y)$
0	0	0.2
0	1	0.3
1	0	0.1
1	1	0.4

A. Mathematics

For example,

$$P(X = 1, Y = 0) = 0.1. \quad (\text{A.23})$$

The normalization condition can be verified directly:

$$0.2 + 0.3 + 0.1 + 0.4 = 1. \quad (\text{A.24})$$

A.2.2. Marginal probability distributions

Often we are interested only in the distribution of a single variable. The probability distribution of one variable alone is obtained by summing over the other variable. This is called the *marginal distribution*.

For discrete variables:

$$P(X = x) = \sum_y P(X = x, Y = y), \quad (\text{A.25})$$

and similarly

$$P(Y = y) = \sum_x P(X = x, Y = y). \quad (\text{A.26})$$

Using the example above:

$$P(X = 0) = P(0, 0) + P(0, 1) = 0.2 + 0.3 = 0.5. \quad (\text{A.27})$$

Likewise,

$$P(Y = 1) = P(0, 1) + P(1, 1) = 0.3 + 0.4 = 0.7. \quad (\text{A.28})$$

A.2.3. Conditional probability

The *conditional probability* describes the probability of one event under the condition that another event has already occurred.

The conditional probability of $X = x$ given $Y = y$ is

$$P(X = x \mid Y = y) = \frac{P(X = x, Y = y)}{P(Y = y)}, \quad (\text{A.29})$$

provided that $P(Y = y) > 0$. Rearranging this expression gives the important relation

$$P(X, Y) = P(X \mid Y) P(Y). \quad (\text{A.30})$$

Similarly,

$$P(X, Y) = P(Y \mid X) P(X). \quad (\text{A.31})$$

These identities are fundamental throughout probability theory and statistics.

Example

Using the previous example,

$$P(X = 1 | Y = 1) = \frac{P(X = 1, Y = 1)}{P(Y = 1)} = \frac{0.4}{0.7} \approx 0.57. \quad (\text{A.32})$$

Thus, if we already know that $Y = 1$, the probability that $X = 1$ increases from 0.5 to approximately 0.57.

A.2.4. Independent random variables

Two random variables are called *independent* if knowledge of one variable does not change the probability distribution of the other. Mathematically, independence means that the joint probability factorizes:

$$P(X, Y) = P(X) P(Y). \quad (\text{A.33})$$

Equivalently,

$$P(X | Y) = P(X). \quad (\text{A.34})$$

If this factorization does not hold, the variables are statistically dependent.

Example

Suppose two fair coins are tossed independently. Define

- $X = 1$ if the first coin shows heads,
- $Y = 1$ if the second coin shows heads.

Then

$$P(X = 1) = \frac{1}{2}, \quad P(Y = 1) = \frac{1}{2}, \quad (\text{A.35})$$

and

$$P(X = 1, Y = 1) = \frac{1}{4} = P(X = 1) P(Y = 1). \quad (\text{A.36})$$

Therefore the variables are independent.

A.2.5. Continuous random variables

For continuous random variables, probabilities at exact points vanish:

$$P(X = x, Y = y) = 0. \quad (\text{A.37})$$

A. Mathematics

Instead we define the *joint probability density function* (PDF)

$$f_{X,Y}(x, y). \quad (\text{A.38})$$

Probabilities are obtained by integration over a region A :

$$P((X, Y) \in A) = \int_A f_{X,Y}(x, y) dx dy. \quad (\text{A.39})$$

For a rectangular region this becomes

$$P(a < X < b, c < Y < d) = \int_a^b \int_c^d f_{X,Y}(x, y) dy dx. \quad (\text{A.40})$$

The normalization condition is

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{X,Y}(x, y) dx dy = 1. \quad (\text{A.41})$$

A.2.6. Marginal distributions for continuous variables

The marginal distributions are obtained by integrating over the other variable:

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dy, \quad (\text{A.42})$$

and

$$f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dx. \quad (\text{A.43})$$

B. Physics

B.1. Canonical ensemble

For the Ising model in thermal equilibrium at temperature T , the system is described by the canonical ensemble. A microstate i with energy E_i has probability

$$P_i = \frac{\exp(-\beta E_i)}{Z}, \quad (\text{B.1})$$

where

$$\beta = \frac{1}{k_B T} \quad (\text{B.2})$$

and

$$Z = \sum_i \exp(-\beta E_i) \quad (\text{B.3})$$

is the partition function.

The ensemble average of the energy is therefore

$$\langle E \rangle = \sum_i E_i P_i = \frac{1}{Z} \sum_i E_i \exp(-\beta E_i). \quad (\text{B.4})$$

B.1.1. Heat capacity

The heat capacity at constant volume is defined as

$$C_V = \frac{\partial \langle E \rangle}{\partial T}. \quad (\text{B.5})$$

Since the Boltzmann factor depends naturally on β , it is useful to first differentiate with respect to β and then use the chain rule:

$$C_V = \frac{\partial \langle E \rangle}{\partial \beta} \frac{\partial \beta}{\partial T}. \quad (\text{B.6})$$

Because

$$\frac{\partial \beta}{\partial T} = -\frac{1}{k_B T^2}, \quad (\text{B.7})$$

we obtain

$$C_V = -\frac{1}{k_B T^2} \frac{\partial \langle E \rangle}{\partial \beta}. \quad (\text{B.8})$$

B.1.2. Derivative of the mean energy

Start from

$$\langle E \rangle = \frac{1}{Z} \sum_i E_i \exp(-\beta E_i). \quad (\text{B.9})$$

Differentiating with respect to β gives

$$\frac{\partial \langle E \rangle}{\partial \beta} = \frac{\partial}{\partial \beta} \left(\frac{1}{Z} \sum_i E_i \exp(-\beta E_i) \right) \quad (\text{B.10})$$

$$= -\frac{1}{Z^2} \frac{\partial Z}{\partial \beta} \sum_i E_i \exp(-\beta E_i) - \frac{1}{Z} \sum_i E_i^2 \exp(-\beta E_i). \quad (\text{B.11})$$

The derivative of the partition function is

$$\frac{\partial Z}{\partial \beta} = - \sum_i E_i \exp(-\beta E_i). \quad (\text{B.12})$$

Rewrite the definition of the mean energy

$$\sum_i E_i \exp(-\beta E_i) = Z \langle E \rangle, \quad (\text{B.13})$$

we can write

$$\frac{\partial Z}{\partial \beta} = -Z \langle E \rangle. \quad (\text{B.14})$$

Inserting this into the derivative of $\langle E \rangle$ yields

$$\frac{\partial \langle E \rangle}{\partial \beta} = -\frac{1}{Z^2} (-Z \langle E \rangle) (Z \langle E \rangle) - \frac{1}{Z} \sum_i E_i^2 \exp(-\beta E_i) \quad (\text{B.15})$$

$$= \langle E \rangle^2 - \langle E^2 \rangle. \quad (\text{B.16})$$

B.1.3. Final result

Substituting this into the definition of C_V gives

$$C_V = \frac{1}{k_B T^2} (\langle E^2 \rangle - \langle E \rangle^2). \quad (\text{B.17})$$

The expression in parentheses is the variance of the energy,

$$\text{Var}(E) = \langle E^2 \rangle - \langle E \rangle^2. \quad (\text{B.18})$$

so

$$C_V = \frac{\text{Var}(E)}{k_B T^2} \quad (\text{B.19})$$

B.1.4. Per spin convention

In simulations one often reports the heat capacity per spin. If the Ising model contains N spins, then

$$c_V = \frac{C_V}{N} = \frac{1}{Nk_B T^2} (\langle E^2 \rangle - \langle E \rangle^2). \quad (\text{B.20})$$

If one uses units with $k_B = 1$, this becomes

$$c_V = \frac{1}{NT^2} (\langle E^2 \rangle - \langle E \rangle^2). \quad (\text{B.21})$$

B.1.5. Physical interpretation

The heat capacity measures how strongly the mean energy reacts to a change in temperature. The equation above shows that this response is equal to the size of the equilibrium energy fluctuations. Therefore, large energy fluctuations imply a large heat capacity.

For the Ising model, these fluctuations become especially large near the critical temperature. This is why the heat capacity develops a pronounced peak in finite systems and diverges in the thermodynamic limit for the two-dimensional Ising model.